

بسمه تعالی

گزارش تحلیل کد: پیاده‌سازی الگوریتم تجزیه Multicut-L-shaped برای حل مسائل برنامه‌ریزی خطی تصادفی دو مرحله‌ای

تهیه کننده: آرمین زینال زاده

۴۰۳۲۰۵۲۳۶

۱. معرفی الگوریتم L-shaped

مسائل برنامه‌ریزی تصادفی دو مرحله‌ای، به ویژه با افزایش تعداد سناریوها، می‌توانند بسیار بزرگ و پیچیده شوند. الگوریتم L-shaped، که نوعی از الگوریتم تجزیه بندرز (Benders Decomposition) است، یک روش کارآمد برای حل این گونه مسائل است. ایده اصلی این الگوریتم، تجزیه مسئله بزرگ به دو بخش است:

۱. **مسئله اصلی (Master Problem):** که فقط شامل متغیرهای مرحله اول است و هزینه مرحله دوم را به صورت تخمینی در نظر می‌گیرد.
۲. **مسائل فرعی (Subproblems):** برای هر سناریو یک مسئله فرعی وجود دارد که با ثابت در نظر گرفتن تصمیمات مرحله اول، هزینه بهینه مرحله دوم را محاسبه می‌کند.

الگوریتم به صورت تکراری بین این دو بخش حرکت می‌کند. با حل مسائل فرعی، اطلاعاتی در قالب "برش" (Cut) به دست می‌آید که به مسئله اصلی اضافه شده و تخمین آن از هزینه مرحله دوم را دقیق‌تر می‌کند. این فرآیند تا زمانی که جواب بهینه پیدا شود، ادامه می‌یابد. کد ارائه شده، این الگوریتم را پیاده‌سازی می‌کند.

۲. تعریف و فرمول‌بندی مسئله

۲.۱. ساختار ریاضی مسئله اصلی (Master Problem)

مسئله اصلی به دنبال یافتن تصمیمات مرحله اول x_1, x_2 است که مجموع هزینه مرحله اول و امید ریاضی هزینه مرحله دوم را کمینه کند.

$$\text{Min } 3x_1 + 2x_2 + \theta_1 + \theta_2$$

- x_1, x_2 : متغیرهای تصمیم مرحله اول هستند.
- θ_s : متغیری است که هزینه بهینه مرحله دوم تحت سناریو s را تخمین می‌زند. در ابتدا این تخمین دقیق نیست و با افزودن برش‌های بهینگی بهبود می‌یابد.

۲.۲. ساختار ریاضی مسئله فرعی (Subproblem)

برای هر سناریو $s \in \{1, 2\}$ ، با داشتن مقادیر x_1, x_2 از مسئله اصلی، یک مسئله فرعی برای یافتن y_1, y_2 حل می‌شود.

$$\text{Min } 15y_1 + 12y_2$$

$$\text{s.t. } 3y_1 + 2y_2 \leq x_1, \quad 2y_1 + 5y_2 \leq x_2$$

$$0.8\xi_1 \leq y_1 \leq \xi_1, \quad 0.8\xi_2 \leq y_2 \leq \xi_2, \quad x, y \geq 0$$

- y_1, y_2 : متغیرهای تصمیم مرحله دوم هستند.
- ξ_{1s}, ξ_{2s} : مقادیر پارامترهای تصادفی در سناریو s هستند که از دیکشنری scenario خوانده می‌شوند.

۳. تحلیل و تشریح کامل کد

در این بخش، اجزای مختلف کد به تفصیل بررسی می‌شوند. کد استفاده شده با نام Multicut-LShape-2Scenario پیوست شده است.

۳.۱. بخش ۱: ورودی‌ها و آماده‌سازی اولیه

```
import numpy as np
from pulp import *
import math

scenario = {
    1 : {"kesi_1" : 4 , "kesi_2" : 4 }
    , 2 : {"kesi_1" : 6 , "kesi_2" : 8}
}
pscenario = [ 0.5 , 0.5]
```

- Import: کتابخانه‌های numpy برای عملیات ماتریسی و pulp برای مدل‌سازی و حل مسائل برنامه‌ریزی خطی استفاده شده‌اند.
- scenario: یک دیکشنری پایتون که پارامترهای تصادفی (ξ) را برای هر سناریو تعریف می‌کند. این دیکشنری در قسمت تولید ۳۰ سناریو، دوباره به شکل دیگری استفاده خواهد شد.
- pscenario: یک لیست که احتمال وقوع هر سناریو را مشخص می‌کند.

۳.۲. بخش ۲: ساخت مسئله اصلی (Master Problem)

```
master = LpProblem('masterproblem' , sense=LpMinimize)
x = LpVariable.dicts('x' , [1,2] , lowBound=0 , cat=LpContinuous )
theta = LpVariable.dicts('theta' , [1,2] , lowBound=-100000 , cat=LpContinuous)
master += 3 * x[1] + 2 * x[2] + theta[1] + theta[2]
master += x[1] + x[2] <= 10000 # manually added constarint
```

- master: مسئله مرحله اول.
- x: متغیرهای تصمیم مرحله اول.
- theta: متغیرهای اضافی تعریف شده برای مرحله اول.

لازم به ذکر است که محدودیت آخر که دارای کامنت می‌باشد، بصورت دستی اضافه شده است و مساله اصلی این محدودیت را ندارد. این محدودیت به این دلیل اضافه شده است که کتابخانه pulp قادر به حل کردن مساله برنامه ریزی خطی بدون محدودیت نمی‌باشد و ارور میدهد. برای رفع این مشکل این محدودیت را اضافه نمودم.

۳.۳. بخش ۳: استخراج ماتریس‌های h و T

از آنجاییکه ماتریس‌های h و T تا انتهای مساله ثابت می‌مانند، یکبار در همین مرحله این ماتریس به ازای هر سناریو استخراج شدند و تا انتهای کد از آنها استفاده شد. ماتریس h ، ضرایب سمت راست مساله مرحله دوم و T ضرایب متغیر x در محدودیت‌های مساله مرحله دوم می‌باشد. برای انجام این کار ابتدا مساله مرحله دوم تعریف شد و سپس با استفاده از کد زیر (همان روشی که در کلاس آنلاین تدریس شده بود) اقدام به استخراج این ماتریس‌ها کردیم.

```
for s in [1,2]:
    kesi_1 = scenario[s]["kesi_1"]
    kesi_2 = scenario[s]["kesi_2"]

    sub = LpProblem('SubProblem', LpMinimize)
    y = LpVariable.dicts('y', [1,2], lowBound=0, cat=LpContinuous)
    sub += 15 * y[1] + 12 * y[2]
    sub += 3 * y[1] + 2 * y[2] - x[1] <= 0
    sub += 2 * y[1] + 5 * y[2] - x[2] <= 0
    sub += y[1] >= 0.8 * kesi_1
    sub += y[2] >= 0.8 * kesi_2
    sub += y[1] <= kesi_1
    sub += y[2] <= kesi_2
```

مساله مرحله دوم تحت عنوان sub تعریف می‌شد و سپس با استفاده از کدی که در تصویر بالا مشهود است، ماتریس‌های h و T را استخراج می‌کنیم. این همان روشی است که در کلاس آنلاین پایتون آموزش داده شد.

۳.۴. تابع $check_cut_feasibility$

این تابع جواب‌های بدست آمده از حل مدل master را بررسی میکند که آیا این جواب در مساله مرحله دوم شدنی است یا نه.

در صورت شدنی بودن مقدار true را برمیگرداند و ادامه ی کد اجرا می‌شود اما در صورت شدنی نبودن یک برش شدنی بودن اضافه میکن و مقدار false را برمیگرداند تا کد دوباره وارد مرحله اول شود.

۱. برای هر سناریو s ، یک (Feasibility Check Problem) تعریف می‌کند. این مسئله با نام w_prime شناخته می‌شود که با نام w' در کتاب به فرم زیر تعریف شده است.

$$\begin{aligned} \min w' &= e^T v^+ + e^T v^- \\ \text{s. t. } Wy + Iv^+ - Iv^- &= h_k - T_k x^v, \\ y &\geq 0, \quad v^+ \geq 0, \quad v^- \geq 0, \end{aligned}$$

۲. اگر مقدار بهینه w_prime بزرگتر از صفر باشد، به این معناست که مسئله فرعی برای x^* داده شده، غیرممکن (infeasible) است.
۳. در این حالت، باید یک (Feasibility Cut) به مسئله اصلی اضافه شود. این برش ناحیه ممکن مسئله اصلی را به گونه‌ای محدود می‌کند که جواب‌های غیرممکن مشابه x^* را در آینده حذف کند.
۴. تابع در صورت افزودن برش، False و در غیر این صورت True برمی‌گرداند.

۳,۵. تابع `check_cut_optimality`

اگر جواب x^* از مسئله اصلی برای تمام سناریوها ممکن (feasible) باشد، باید بررسی کنیم که آیا جواب بدست آمده بهینه می‌باشد و یا خیر. مانند تابع تعریف شده در قسمت قبل، این تابع نیز با همان منطق کد شده است.

۱. برای هر سناریو s ، مسئله فرعی (sub) را با مقدار x^* حل می‌کند و بررسی می‌کند که آیا جواب بهینه است یا نه. اگر جواب بهینه نبود یک برش بهینگی (Optimality Cut) به مسئله اصلی اضافه می‌شود. این برش، کران پایین متغیر θ_s را بهبود می‌بخشد.
۲. تابع در صورت افزودن برش، False و در غیر این صورت True برمی‌گرداند.

۳,۶. حلقه اصلی الگوریتم و شرایط توقف

این الگوریتم در اصل ۳ شرط توقف دارد. اولی وقتی مساله مرحله اول حل می‌شود. دومی وقتی شدنی بودن آن بررسی و برش شدنی بودن اضافه می‌شود و بعد از تکرار مرحله اول جواب شدنی بدست می‌آید. سومی زمانی اتفاق می‌افتد که بهینگی جواب بررسی و برش بهینگی اضافه می‌شود و پس از برگشت مجدد به مرحله اول و سپس بررسی مجدد شدنی بودن آن، جواب بهینه بدست می‌آید و شرط توقف سوم نیز True می‌شود. منطق و فرم کلی الگوریتم بصورت زیر می‌باشد :

۱. **حلقه بیرونی:** تا زمانی که به جواب بهینه کلی نرسیده‌ایم ادامه دارد (3_STOP_COND برابر False است).
۲. **حلقه درونی:** ابتدا روی پیدا کردن یک جواب ممکن تمرکز می‌کند.
 - مسئله اصلی حل می‌شود.
 - تابع `check_cut_feasibility` فراخوانی می‌شود.
 - اگر جواب ناممکن بود، یک برش امکان‌پذیری اضافه شده و حلقه درونی دوباره تکرار می‌شود.
 - این حلقه تا زمانی ادامه می‌یابد که یک x پیدا شود که برای تمام سناریوها ممکن باشد.

```

STOP_COND_1 = False
STOP_COND_2 = False
STOP_COND_3 = False
FEASIBILITY = False
OPTIMALITY = False

while not STOP_COND_3:
    STOP_COND_2 = False
    STOP_COND_1 = False

    while not STOP_COND_2:
        master.solve(PULP_CBC_CMD(msg=False))
        print(f" x1 is : {x[1].varValue} , x2 is : {x[2].varValue}")
        STOP_COND_1 = True

        FEASIBILITY = check_cut_feasibility(x, master, h, T, scenario)
        print(f"\nRESULT OF FEASIBILITY: {FEASIBILITY}")

        if FEASIBILITY:
            STOP_COND_2 = True
        else:
            print("ADDIN FEASIBILITY CUT ...")
            STOP_COND_2 = False
            STOP_COND_1 = False
        print("\n", '***' * 15, "\n")

    OPTIMALITY = check_cut_optimality(x, master, h, T, scenario, theta)
    print(f"\nRESULT OF OPTIMALITY: {OPTIMALITY}")

    if OPTIMALITY:
        STOP_COND_3 = True
    else:
        print("ADDIN OPTIMALITY CUT ...")
        STOP_COND_3 = False
        STOP_COND_2 = False
        STOP_COND_1 = False
    print("\n", '***' * 15, "\n")

print(f"FINAL SOLUTION: X1: {x[1].varValue}, X2: {x[2].varValue}")

```

۳. بررسی بهینگی: پس از یافتن یک جواب ممکن، `check_cut_optimality` فراخوانی می‌شود.

- اگر شرط بهینگی `false` بود، یک برش بهینگی اضافه می‌شود و الگوریتم به ابتدای حلقه بیرونی بازمی‌گردد تا مسئله اصلی را با این برش جدید دوباره حل کند.

- اگر جواب هم ممکن و هم بهینه بود (یعنی هیچ برش جدیدی از هیچ نوعی اضافه نشد)، هر دو تابع `check_cut_optimality` و `check_cut_feasibility` مقدار `True` برمی‌گردانند، `STOP_COND_3` برابر `True` شده و الگوریتم متوقف می‌شود.

۴. روند اجرا و تحلیل خروجی

```
1 | x1 is : 0.0 , x2 is : 0.0
2 | added feasibility cut to master problem : 0.27272727*x_1 + 0.090909091*x_2 >= 6.4
3 | added feasibility cut to master problem : 0.27272727*x_1 + 0.090909091*x_2 >= 11.200000000000001
4 |
5 | RESULT OF FEASIBILITY: False
6 | ADDIN FEASIBILITY CUT ...
7 |
8 | *****
9 |
10 | x1 is : 41.066667 , x2 is : 0.0
11 | added feasibility cut to master problem : 0.5*x_2 >= 6.4
12 | added feasibility cut to master problem : 0.5*x_2 >= 11.200000000000001
13 |
14 | RESULT OF FEASIBILITY: False
15 | ADDIN FEASIBILITY CUT ...
16 |
17 | *****
18 |
19 | x1 is : 33.6 , x2 is : 22.4
20 | added feasibility cut to master problem : 0.2*x_2 >= 8.32
21 |
22 | RESULT OF FEASIBILITY: False
23 | ADDIN FEASIBILITY CUT ...
24 |
25 | *****
26 |
27 | x1 is : 27.2 , x2 is : 41.6
28 |
29 | RESULT OF FEASIBILITY: True
30 |
31 | *****
32 |
33 | added optimally cut to master problem : theta_1 >= -48.0
34 | added optimally cut to master problem : theta_2 >= -72.0
35 |
36 | RESULT OF OPTIMALITY: False
37 | ADDIN OPTIMALITY CUT ...
38 |
39 | *****
40 |
41 | x1 is : 27.2 , x2 is : 41.6
42 |
43 | RESULT OF FEASIBILITY: True
44 |
45 | *****
46 |
47 | RESULT OF OPTIMALITY: True
48 |
49 | *****
50 |
51 |
52 | FINAL SOLUTION: X1: 27.2, X2: 41.6
53 |
```

در این تکرار جواب شدنی بدست می‌آید.

برش بهینگی برای هر تتا جداگانه اضافه می‌شود.

جواب هم شدنی و هم بهینه می‌باشد پس الگوریتم خاتمه می‌یابد.

الگوریتم شروع به اجرا می‌شود، در تکرار اول دو برش شدنی بودن (که یکی مازاد است) به مساله ی master اضافه می‌شود. دوباره مساله ی master حل می‌شود. این روند تا ۳ تکرار ادامه پیدا میکند تا اینکه جواب بدست آمده شدنی باشد. سپس جواب بدست آمده وارد مرحله سوم می‌شود و پس از اضافه کردن برش بهینگی، جواب بهینه بدست می‌آید. خروجی ارائه شده روند کامل الگوریتم را نشان می‌دهد.

۵. تولید ۳۰ سناریو و اجرای الگوریتم

برای انجام این قسمت، سناریوی تعریف شده، بردار احتمالات، و تمامی توابعی که به ازای هر سناریو اجرا می‌شدند دستخوش تغییرات شدند. بدین گونه که برای تعریف سناریو (تولید سناریوهای تصادفی) از کد زیر استفاده شد.

فایل این قسمت با نام Multicut-LShape-MultiScenario پیوست شده است.

```
np.random.seed(42)

num_scenarios = 30
scenario = {s: {"kesi_1": np.random.randint(2, 10), "kesi_2": np.random.randint(2, 12)} for s in range(1, num_scenarios + 1)}

# Assume equal probability for each scenario
pscenario = [1.0 / num_scenarios] * num_scenarios
```

در این قسمت یک دیکشنری به نام سناریو تولید شده است که حاوی ۳۰ سناریو می‌باشد. هر سناریو دو مقدار تصادفی دارد که با استفاده از تابع `np.randint (a, b)` تولید شده‌اند. احتمال هر سناریو نیز یکسان در نظر گرفته شده است.

تمامی حلقه‌هایی که در قسمت اول کد برای ۲ تکرار تعریف شده بودند، در این قسمت برای تعداد سناریوهای تعریف شدند و در جواب بهینه بدست آمد. خروجی این قسمت از کد بسیار بزرگ است (از آنجاییکه در هر تکرار ۳۰ برش به مساله اضافه می‌کند و در خروجی چاپ می‌کند) به همین علت فقط دو تکرار به عنوان قسمتی از خروجی را در این گزارش ارائه مینمایم.

قسمت اول مربوط به برش‌های شدنی شدن و تصویر دوم مربوط به خروجی برش‌های بهینگی می‌باشد.


```
x1 is : 55.733334 , x2 is : 0.0
```

```
added feasibility cut to master problem : 0.5*x_2 >= 10.4
added feasibility cut to master problem : 0.5*x_2 >= 12.0
added feasibility cut to master problem : 0.5*x_2 >= 9.600000000000001
added feasibility cut to master problem : 0.5*x_2 >= 15.200000000000001
added feasibility cut to master problem : 0.5*x_2 >= 9.600000000000001
added feasibility cut to master problem : 0.5*x_2 >= 10.4
added feasibility cut to master problem : 0.5*x_2 >= 8.8
added feasibility cut to master problem : 0.5*x_2 >= 14.4
added feasibility cut to master problem : 0.5*x_2 >= 8.8
added feasibility cut to master problem : 0.5*x_2 >= 7.200000000000001
added feasibility cut to master problem : 0.5*x_2 >= 12.8
added feasibility cut to master problem : 0.5*x_2 >= 7.200000000000001
added feasibility cut to master problem : 0.5*x_2 >= 10.4
added feasibility cut to master problem : 0.5*x_2 >= 13.600000000000001
added feasibility cut to master problem : 0.5*x_2 >= 10.4
added feasibility cut to master problem : 0.5*x_2 >= 10.4
added feasibility cut to master problem : 0.5*x_2 >= 10.4
added feasibility cut to master problem : 0.5*x_2 >= 12.0
added feasibility cut to master problem : 0.5*x_2 >= 8.0
added feasibility cut to master problem : 0.5*x_2 >= 9.600000000000001
added feasibility cut to master problem : 0.5*x_2 >= 12.8
added feasibility cut to master problem : 0.5*x_2 >= 8.8
added feasibility cut to master problem : 0.5*x_2 >= 12.0
added feasibility cut to master problem : 0.5*x_2 >= 6.4
added feasibility cut to master problem : 0.5*x_2 >= 10.4
added feasibility cut to master problem : 0.5*x_2 >= 7.200000000000001
added feasibility cut to master problem : 0.5*x_2 >= 6.4
added feasibility cut to master problem : 0.5*x_2 >= 10.4
added feasibility cut to master problem : 0.5*x_2 >= 11.2
added feasibility cut to master problem : 0.5*x_2 >= 9.600000000000001
```

```
RESULT OF FEASIBILITY: False
ADDIN FEASIBILITY CUT ...
```

```
*****
```

قابل توجه است که یکی از این برش ها به عنوان محدودیت فعال عمل میکند و مابقی محدودیت های اضافی هستند، اما برای نشان دادن نحوه عملکرد الگوریتم و نیز اجتناب از پیچیدگی به همین منوال ادامه دادم. در صورت نیاز میتوان فقط برش فعال را به مساله مرحله اول اضافه کرد.

قسمت دوم از نمونه خروجی کد مربوط به برش های بهینگی می باشد که در هر تکرار ۳۰ برش به مساله مرحله اضافه میکند.

```
added optimally cut to master problem : theta_1 >= -6.4
added optimally cut to master problem : theta_2 >= -4.800000000000001
added optimally cut to master problem : theta_3 >= -4.800000000000001
added optimally cut to master problem : theta_4 >= -6.4
added optimally cut to master problem : theta_5 >= -3.2
added optimally cut to master problem : theta_6 >= -3.2
added optimally cut to master problem : theta_7 >= -4.800000000000001
added optimally cut to master problem : theta_8 >= -7.2
added optimally cut to master problem : theta_9 >= -3.2
added optimally cut to master problem : theta_10 >= -4.800000000000001
added optimally cut to master problem : theta_11 >= -7.2
added optimally cut to master problem : theta_12 >= -2.4000000000000004
added optimally cut to master problem : theta_13 >= -1.6
added optimally cut to master problem : theta_14 >= -5.6000000000000005
added optimally cut to master problem : theta_15 >= -1.6
added optimally cut to master problem : theta_16 >= -7.2
added optimally cut to master problem : theta_17 >= -4.0
added optimally cut to master problem : theta_18 >= -4.0
added optimally cut to master problem : theta_19 >= -3.2
added optimally cut to master problem : theta_20 >= -3.2
added optimally cut to master problem : theta_21 >= -4.800000000000001
added optimally cut to master problem : theta_22 >= -6.4
added optimally cut to master problem : theta_23 >= -4.0
added optimally cut to master problem : theta_24 >= -4.0
added optimally cut to master problem : theta_25 >= -2.4000000000000004
added optimally cut to master problem : theta_26 >= -2.4000000000000004
added optimally cut to master problem : theta_27 >= -2.4000000000000004
added optimally cut to master problem : theta_28 >= -4.0
added optimally cut to master problem : theta_29 >= -4.0
added optimally cut to master problem : theta_30 >= -6.4

RESULT OF OPTIMALITY: False
ADDIN OPTIMALITY CUT ...
```

برای انجام این پروژه، ابتدا منطق و ساختار کلی کد طراحی شد (بخش ۳،۶ – برای ارائه cleancode) و سپس اقدام به تعریف فانکشن ها نمودم. برای تعاریف و نوشتن کد مربوط به فانکشن ها از نمونه کد های تدریس شده استفاده بسیار کردم اما در مواردی ارور هایی دریافت میکردم که قادر به حل آن ها نبودم. در این شرایط از chatgpt استفاده میکردم به گونه ای که آن قسمت از کدی که منجر به ارور میشد و توضیحات ارور را به عنوان prompt میدادم. لازم به ذکر است هیچ قسمتی از کد وجود ندارد که صرفا توسط هوش مصنوعی نوشته شده باشد.